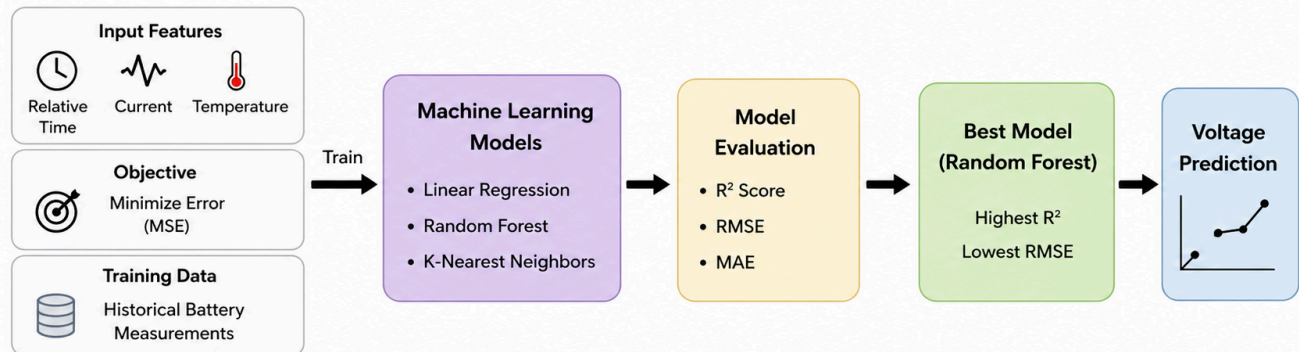


Battery Voltage Prediction Using Machine Learning

Colin Fields Tierney Lawrence

University at Albany
Machine Learning Project 2026

[Code](#)[View PDF](#)

Our machine learning pipeline uses time, current, and temperature as input features to train multiple models and predict battery voltage.

Abstract

This project explores the use of machine learning techniques to predict battery voltage behavior using real-world charge and discharge cycle data. The dataset consists of approximately 7.2 million observations and includes key features such as relative time, current, and temperature. Multiple models were implemented and evaluated, including linear regression, random forest, K-nearest neighbors (KNN), and a neural network. Performance was assessed using RMSE and R^2 metrics. Results show that nonlinear models, particularly random forest and neural networks, significantly outperform linear approaches.

Key Observations

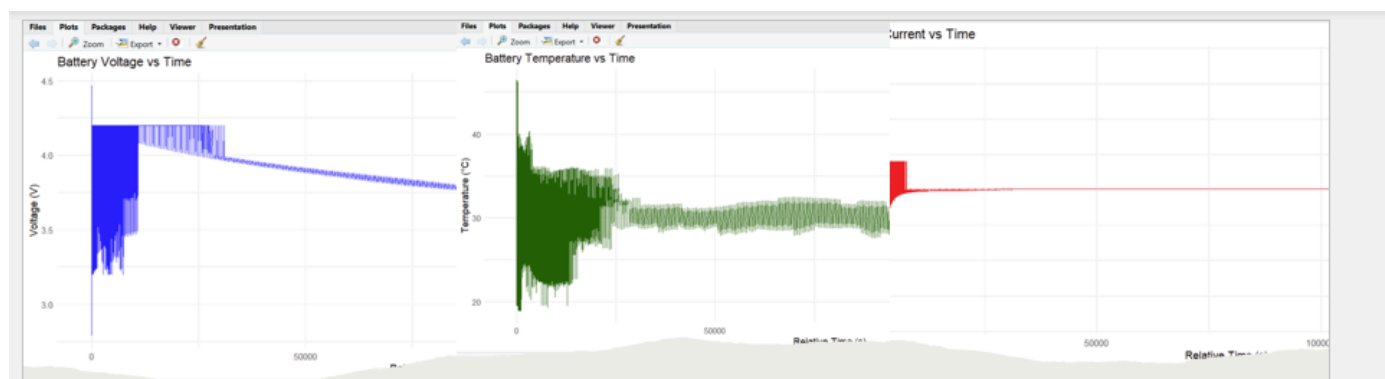


Figure 1: Battery behavior over time.

Analysis of the dataset reveals several important patterns in battery behavior. Voltage fluctuates during charge and discharge cycles, while current exhibits spikes during load changes. Temperature varies gradually based on battery activity. These patterns indicate that relationships between variables are nonlinear, motivating the use of advanced machine learning models.

Dataset

comment	type	time	relativeTime	voltage	current	temperature	dat
1 reference charge	C	c[" 6.62", " 16.62", " 26.62", " 36.62", " 4 [...]"]	c[" 0.01", " 10.01", " 20.01", " 30.01", " 4 [...]"]	c["4.196", "4.200", "4.200", "4.200", "4.200", "4 [...]"]	c[" 0.000", "-0.039", "-0.035", "-0.033", "-0.032 [...]"]	c["22.39476", "22.41031", "22.30144", "22.27033", [...]"]	
2 reference discharge	D	c["1633.73", "1643.73", "1653.73", "1663.73", "167 [...]"]	c[" 0.04", " 10.04", " 20.04", " 30.04", " 4 [...]"]	c["4.116", "4.102", "4.094", "4.088", "4.082", "4 [...]"]	c["1.005", "1.000", "1.000", "1.000", "1.000", "1 [...]"]	c["22.72138", "20.09283", "20.10838", "20.12394", [...]"]	
3 reference charge	C	c[" 9176.69", " 9186.69", " 9196.69", " 9206.69", [...]"]	c[" 0.05", " 10.05", " 20.05", " 30.05", " [...]"]	c["3.494", "3.601", "3.650", "3.690", "3.722", "3 [...]"]	c["-2.004", "-2.000", "-2.000", "-2.000", "-2.000 [...]"]	c["26.36092", "26.36092", "26.36092", "26.32982", [...]"]	
4 reference discharge	D	c["19672.49", "19682.49", "19692.49", "19702.49", [...]"]	c[" 0.04", " 10.04", " 20.04", " 30.04", " 4 [...]"]	c["4.116", "4.103", "4.094", "4.088", "4.082", "4 [...]"]	c["1.001", "0.999", "1.000", "1.000", "1.000", "1 [...]"]	c["23.21910", "23.23465", "23.26576", "23.29687", [...]"]	
5 pulsed load (rest)	R	c["37626.98", "37627.98", "37628.98", "37629.98", [...]"]	c[" 0.01", " 1.01", " 2.01", " 3.01", " [...]"]	c["4.199", "4.199", "4.199", "4.199", "4.199", "4 [...]"]	c["0", "0", "0", "0", "0", "0", "0", "0", "0", "0 [...]"]	c["22.98580", "22.98580", "22.98580", "22.98580", [...]"]	
6 pulsed load (discharge)	D	c["38827.01", "38828.01", "38829.01", "38830.01", [...]"]	c[" 0.04", " 1.04", " 2.04", " 3.04", " 4.04 [...]"]	c["4.114", "4.110", "4.109", "4.107", "4.106", "4 [...]"]	c["1.003", "1.000", "1.000", "1.000", "0.999", "1 [...]"]	c["22.87692", "22.87692", "22.87692", "22.87692", [...]"]	
7 pulsed load (rest)	R	c["39426.98", "39427.98", "39428.98", "39429.98", [...]"]	c[" 0.01", " 1.01", " 2.01", " 3.01", " [...]"]	c["4.061", "4.069", "4.071", "4.073", "4.074", "4 [...]"]	c["0", "0", "0", "0", "0", "0", "0", "0", "0", "0 [...]"]	c["23.57683", "23.57683", "23.57683", "23.59239", [...]"]	
8 pulsed load (discharge)	D	c["40627.01", "40628.01", "40629.01", "40630.01", [...]"]	c[" 0.04", " 1.04", " 2.04", " 3.04", " 4.04 [...]"]	c["4.036", "4.032", "4.030", "4.028", "4.027", "4 [...]"]	c["1.004", "1.000", "1.000", "0.999", "1.001", "1 [...]"]	c["20.27947", "20.27947", "20.27947", "20.27947", [...]"]	
9 pulsed load (rest)	R	c["41226.98", "41227.98", "41228.98", "41229.98", [...]"]	c[" 0.01", " 1.01", " 2.01", " 3.01", " [...]"]	c["3.987", "3.995", "3.998", "3.999", "4.000", "4 [...]"]	c["0", "0", "0", "0", "0", "0", "0", "0", "0", "0 [...]"]	c["20.93272", "20.93272", "20.94828", "20.94828", [...]"]	
10 pulsed load (discharge)	D	c["42427.01", "42428.01", "42429.01", "42430.01", [...]"]	c[" 0.04", " 1.04", " 2.04", " 3.04", " 4.04 [...]"]	c["3.969", "3.964", "3.962", "3.961", "3.959", "3 [...]"]	c["1.003", "1.000", "1.000", "1.000", "0.999", "1 [...]"]	c["23.26576", "23.26576", "23.26576", "23.26576", [...]"]	
11 pulsed load (rest)	R	c["43026.98", "43027.98", "43028.98", "43029.98", [...]"]	c[" 0.01", " 1.01", " 2.01", " 3.01", " [...]"]	c["3.922", "3.930", "3.932", "3.933", "3.935", "3 [...]"]	c["0", "0", "0", "0", "0", "0", "0", "0", "0", "0 [...]"]	c["23.84124", "23.85680", "23.84124", "23.85680", [...]"]	
12 pulsed load (discharge)	D	c["44227.01", "44228.01", "44229.01", "44230.01", [...]"]	c[" 0.04", " 1.04", " 2.04", " 3.04", " 4.04 [...]"]	c["3.911", "3.906", "3.904", "3.902", "3.900", "3 [...]"]	c["1.003", "1.000", "1.000", "1.000", "1.000", "1 [...]"]	c["22.95469", "22.95469", "22.95469", "22.95469", [...]"]	
13 pulsed load (rest)	R	c["44826.98", "44827.98", "44828.98", "44829.98", [...]"]	c[" 0.01", " 1.01", " 2.01", " 3.01", " [...]"]	c["3.866", "3.874", "3.875", "3.877", "3.878", "3 [...]"]	c["0", "0", "0", "0", "0", "0", "0", "0", "0", "0 [...]"]	c["21.29045", "21.29045", "21.29045", "21.29045", [...]"]	
14 pulsed load (discharge)	D	c["46027.01", "46028.01", "46029.01", "46030.01", [...]"]	c[" 0.04", " 1.04", " 2.04", " 3.04", " 4.04 [...]"]	c["3.861", "3.856", "3.854", "3.852", "3.850", "3 [...]"]	c["1.002", "1.000", "1.000", "1.001", "1.000", "1 [...]"]	c["23.15689", "23.15689", "23.15689", "23.15689", [...]"]	
15 pulsed load (rest)	R	c["46626.98", "46627.98", "46628.98", "46629.98", [...]"]	c[" 0.01", " 1.01", " 2.01", " 3.01", " [...]"]	c["3.821", "3.828", "3.829", "3.830", "3.831", "3 [...]"]	c["0", "0", "0", "0", "0", "0", "0", "0", "0", "0 [...]"]	c["23.84124", "23.84124", "23.84124", "23.84124", [...]"]	
16 pulsed load (discharge)	D	c["47827.01", "47828.01", "47829.01", "47830.01", [...]"]	c[" 0.04", " 1.04", " 2.04", " 3.04", " 4.04 [...]"]	c["3.809", "3.804", "3.802", "3.800", "3.798", "3 [...]"]	c["1.006", "0.999", "1.000", "1.000", "1.000", "1 [...]"]	c["23.21910", "23.21910", "23.21910", "23.21910", [...]"]	
17 pulsed load (rest)	R	c["48426.98", "48427.98", "48428.98", "48429.98", [...]"]	c[" 0.01", " 1.01", " 2.01", " 3.01", " [...]"]	c["3.783", "3.790", "3.791", "3.792", "3.794", "3 [...]"]	c["0", "0", "0", "0", "0", "0", "0", "0", "0", "0 [...]"]	c["23.67015", "23.67015", "23.67015", "23.68571", [...]"]	
18 pulsed load (discharge)	D	c["49627.01", "49628.01", "49629.01", "49630.01", [...]"]	c[" 0.04", " 1.04", " 2.04", " 3.04", " 4.04 [...]"]	c["3.754", "3.751", "3.750", "3.749", "3.747", "3 [...]"]	c["1.003", "1.000", "1.000", "1.000", "0.999", "0 [...]"]	c["22.42587", "22.42587", "22.44142", "22.44142", [...]"]	
19 pulsed load (rest)	R	c["50226.98", "50227.98", "50228.98", "50229.98", [...]"]	c[" 0.01", " 1.01", " 2.01", " 3.01", " [...]"]	c["3.752", "3.759", "3.761", "3.762", "3.763", "3 [...]"]	c["0", "0", "0", "0", "0", "0", "0", "0", "0", "0 [...]"]	c["23.46796", "23.46796", "23.46796", "23.46796", [...]"]	
20 pulsed load (discharge)	D	c["51427.01", "51428.01", "51429.01", "51430.01", [...]"]	c[" 0.04", " 1.04", " 2.04", " 3.04", " 4.04 [...]"]	c["3.724", "3.722", "3.721", "3.719", "3.718", "3 [...]"]	c["1.003", "1.000", "1.000", "1.000", "1.000", "0 [...]"]	c["20.32613", "20.32613", "20.32613", "20.31058", [...]"]	
21 pulsed load (rest)	R	c["52026.98", "52027.98", "52028.98", "52029.98", [...]"]	c[" 0.01", " 1.01", " 2.01", " 3.01", " [...]"]	c["3.725", "3.733", "3.734", "3.736", "3.737", "3 [...]"]	c["0", "0", "0", "0", "0", "0", "0", "0", "0", "0 [...]"]	c["23.95012", "23.93456", "23.93456", "23.91901", [...]"]	
22 pulsed load (discharge)	D	c["53227.01", "53228.01", "53229.01", "53230.01", [...]"]	c[" 0.04", " 1.04", " 2.04", " 3.04", " 4.04 [...]"]	c["3.704", "3.701", "3.700", "3.699", "3.698", "3 [...]"]	c["1.002", "1.000", "1.000", "1.001", "1.000", "1 [...]"]	c["23.06356", "23.06356", "23.06356", "23.06356", [...]"]	
23 pulsed load (rest)	R	c["53826.98", "53827.98", "53828.98", "53829.98", [...]"]	c[" 0.01", " 1.01", " 2.01", " 3.01", " [...]"]	c["3.703", "3.711", "3.713", "3.714", "3.715", "3 [...]"]	c["0", "0", "0", "0", "0", "0", "0", "0", "0", "0 [...]"]	c["24.15232", "24.15232", "24.15232", "24.15232", [...]"]	
24 pulsed load (discharge)	D	c["55027.01", "55028.01", "55029.01", "55030.01", [...]"]	c[" 0.04", " 1.04", " 2.04", " 3.04", " 4.04 [...]"]	c["3.690", "3.687", "3.686", "3.685", "3.684", "3 [...]"]	c["1.002", "1.000", "0.999", "1.000", "1.000", "0 [...]"]	c["23.01690", "23.01690", "23.01690", "23.01690", [...]"]	
25 pulsed load (rest)	R	c["55626.98", "55627.98", "55628.98", "55629.98", [...]"]	c[" 0.01", " 1.01", " 2.01", " 3.01", " [...]"]	c["3.679", "3.689", "3.690", "3.691", "3.693", "3 [...]"]	c["0", "0", "0", "0", "0", "0", "0", "0", "0", "0 [...]"]	c["24.38562", "24.38562", "24.38562", "24.38562", [...]"]	
26 pulsed load (discharge)	D	c["56827.01", "56828.01", "56829.01", "56830.01", [...]"]	c[" 0.04", " 1.04", " 2.04", " 3.04", " 4.04 [...]"]	c["3.671", "3.668", "3.666", "3.665", "3.664", "3 [...]"]	c["1.003", "0.999", "1.000", "1.000", "1.000", "1 [...]"]	c["23.28131", "23.28131", "23.28131", "23.28131", [...]"]	
27 pulsed load (rest)	R	c["57426.98", "57427.98", "57428.98", "57429.98", [...]"]	c[" 0.01", " 1.01", " 2.01", " 3.01", " [...]"]	c["3.644", "3.655", "3.656", "3.657", "3.658", "3 [...]"]	c["0", "0", "0", "0", "0", "0", "0", "0", "0", "0 [...]"]	c["24.68114", "24.68114", "24.68114", "24.68114", [...]"]	
28 pulsed load (discharge)	D	c["58627.01", "58628.01", "58629.01", "58630.01", [...]"]	c[" 0.04", " 1.04", " 2.04", " 3.04", " 4.04 [...]"]	c["3.616", "3.612", "3.611", "3.609", "3.608", "3 [...]"]	c["1.002", "1.000", "1.000", "1.000", "1.000", "1 [...]"]	c["23.56128", "23.56128", "23.56128", "23.54572", [...]"]	
29 pulsed load (rest)	R	c["59226.98", "59227.98", "59228.98", "59229.98", [...]"]	c[" 0.01", " 1.01", " 2.01", " 3.01", " [...]"]	c["3.578", "3.596", "3.598", "3.599", "3.601", "3 [...]"]	c["0", "0", "0", "0", "0", "0", "0", "0", "0", "0 [...]"]	c["24.68114", "24.68114", "24.68114", "24.68114", [...]"]	

Figure 2: Dataset distribution.

The dataset contains approximately 7.2 million observations collected from randomized battery usage cycles. It includes relative time, voltage, current, and temperature as features. The data was preprocessed to ensure consistency, including converting values to numeric format and organizing structured data. This resulted in a clean dataset suitable for training machine learning models.

Results

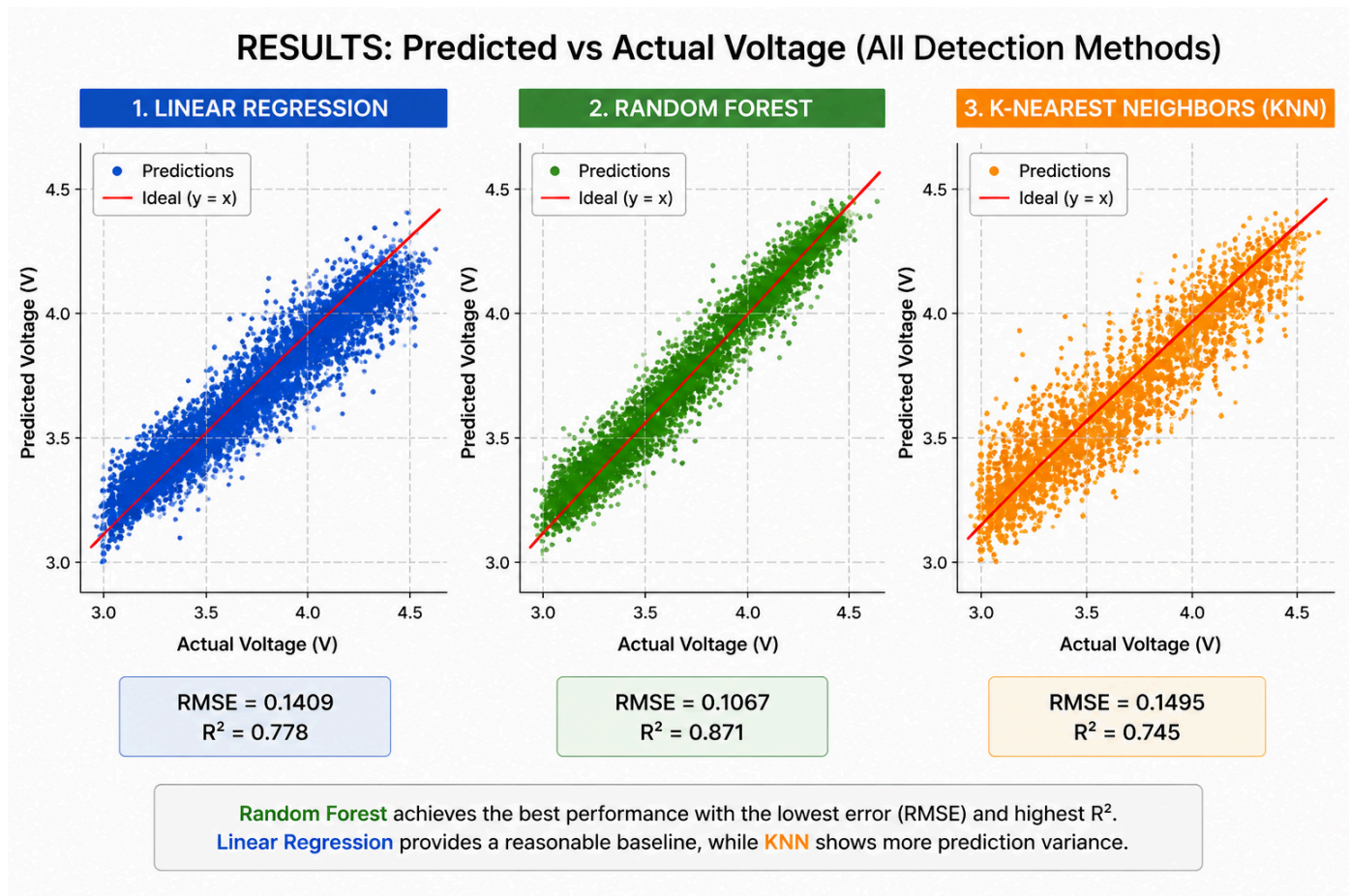


Figure 3: Predicted vs actual voltage.

Four models were evaluated: linear regression, random forest, KNN, and a neural network. Linear regression achieved moderate performance but was limited by its linear assumptions. The random forest model achieved the best performance with an RMSE of approximately 0.107 and R² of 0.87. The neural network performed similarly, while KNN showed higher error and variability. These results confirm that nonlinear models significantly improve prediction accuracy.

Neural Network Model

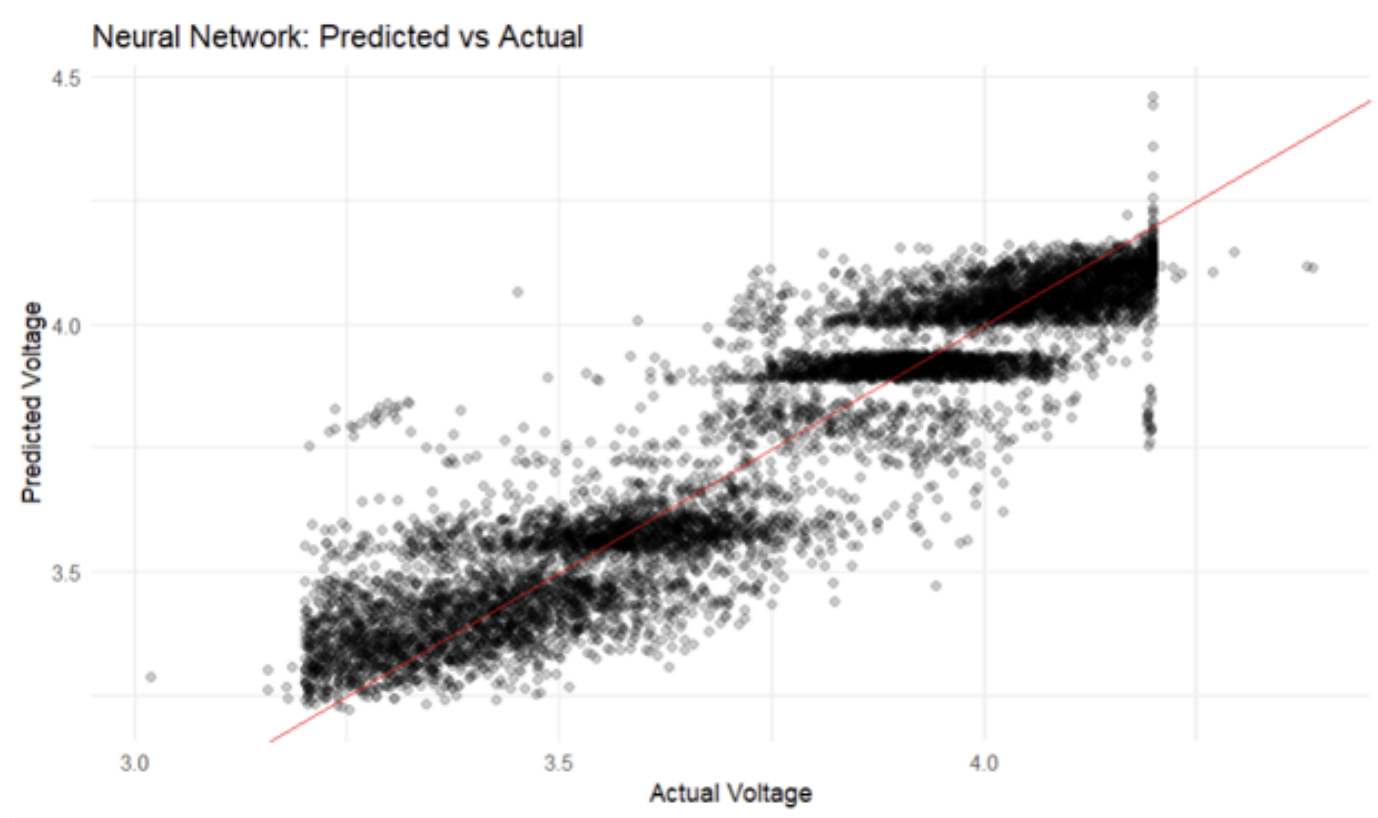


Figure 4: Neural network predictions.

A feedforward neural network was implemented to model nonlinear relationships in the data. The model achieved an RMSE of approximately 0.110 and R^2 of 0.863, performing close to the random forest model. This demonstrates that neural networks are highly effective for predicting battery behavior when sufficient data is available.

Improvements

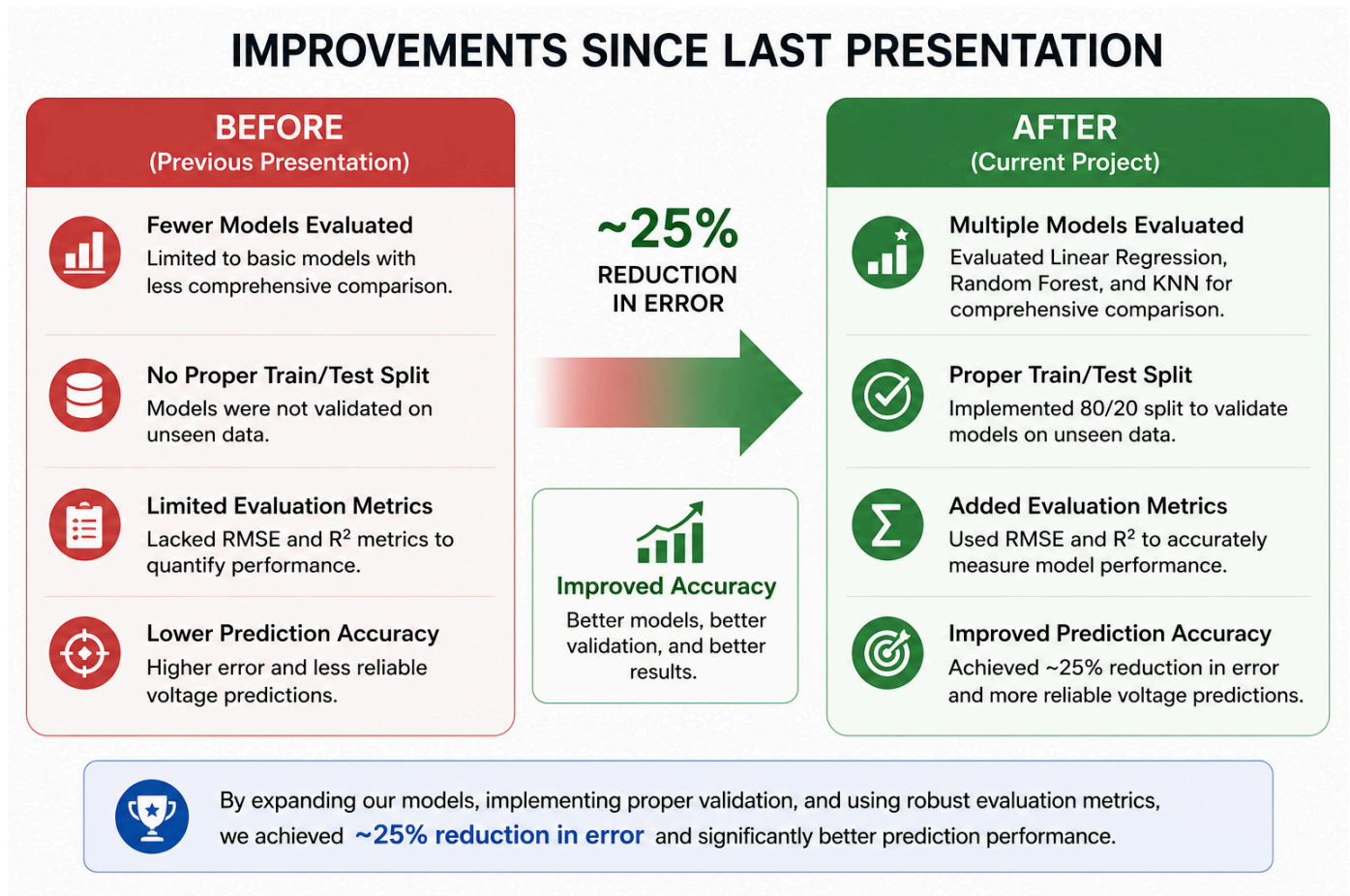


Figure 5: Model improvements.

Significant improvements were made compared to earlier stages of the project. Additional models were introduced, proper train-test splitting was implemented, and evaluation metrics were added. These improvements resulted in approximately a 25% reduction in prediction error and a substantial increase in model reliability.

Model Performance

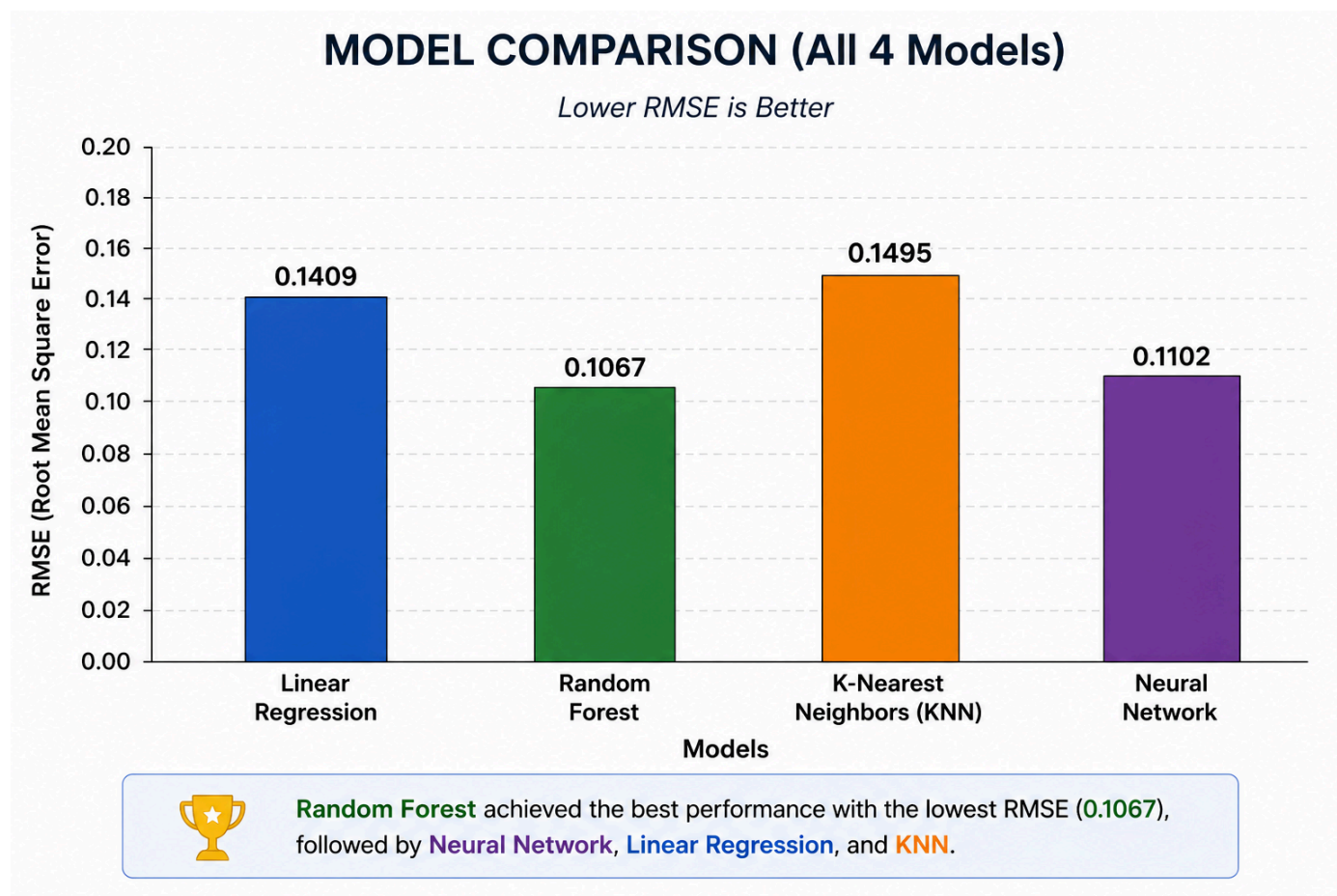


Figure 6: Model comparison.

Model performance was evaluated using RMSE and R^2 metrics. Random forest achieved the best overall performance, followed closely by the neural network. Linear regression served as a baseline, while KNN showed the highest error. These results highlight the importance of using nonlinear models for complex datasets.